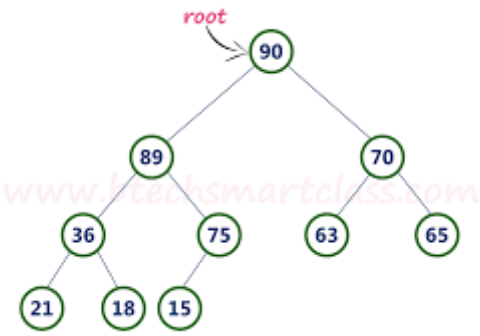
Max Heap and Min heap in Binary tree

Definition

A **Max Heap** is a special type of **complete binary tree** in which:

- Every **parent node is greater than or equal to its children**.
- The **largest element is always at the root node**.
- The tree is **complete**, meaning all levels are filled except possibly the last, which is filled from **left to right**.



2. Properties of Max Heap

1. **Heap Property**
   For every node i

   Parent(i)≥LeftChild(i),RightChild(i)Parent(i) \ge LeftChild(i), RightChild(i)Parent(i)≥LeftChild(i),RightChild(i)

2. **Complete Binary Tree Property**
   - Nodes are filled **level by level**.
   - The last level is filled **from left to right**.

3. Example

```
      90
     /  \
   70    60
  / \    /
 40 50  30
```

Here:

- 90 ≥ 70, 60
- 70 ≥ 40, 50
- 60 ≥ 30

So it satisfies **Max Heap property**.

4. Array Representation of Max Heap

Max heaps are usually stored in **arrays**.

If index of node = i

Node	Formula
Parent	(i-1) / 2
Left Child	2i + 1
Right Child	2i + 2

Example array representation:

Tree:

```
      90
     /  \
    70    60
   / \    /
  40 50  30
```

Array:
[90, 70, 60, 40, 50, 30]

5. Operations in Max Heap

1. **Insertion**
   - Insert element at the **last position**
   - Compare with parent
   - Swap until heap property satisfied (called **Heapify Up**)

2. **Delete a Node from a Max Heap**

In a **Max Heap**, deletion usually means **deleting the root element (maximum element)**.

Steps for Deletion

1. Store the **root element** (element to be deleted).
2. Replace the root with the **last element** in the heap.
3. Reduce the heap size.
4. Perform **Heapify Down** (swap with the larger child until heap property is restored).

3. **Heapify**
   - Used to maintain heap structure.

6. Example of Insertion

Insert **80**

Before:

```
      90
     /  \
   70    60
  / \    /
 40 50  30
```

Insert at last:

```
      90
     /  \
   70    60
  / \    / \
 40 50  30 80
```

Swap with parent:

```
      90
     /  \
   70    80
  / \    / \
 40 50  30 60
```

Heap property satisfied.

7. Applications of Max Heap

Applications of Heap (Binary Heap)

A **Heap** is a **complete binary tree** that satisfies either the **Max Heap** property (parent ≥ children) or the **Min Heap** property (parent ≤ children).
Because heaps allow **fast insertion and deletion of the highest/lowest priority element**, they are widely used in many computer science applications.

1. Priority Queue

A **Priority Queue** is one of the most common applications of a heap.

- In a **Max Heap**, the element with the **highest priority (largest value)** is removed first.
- In a **Min Heap**, the element with the **lowest value** is removed first.

Example Applications

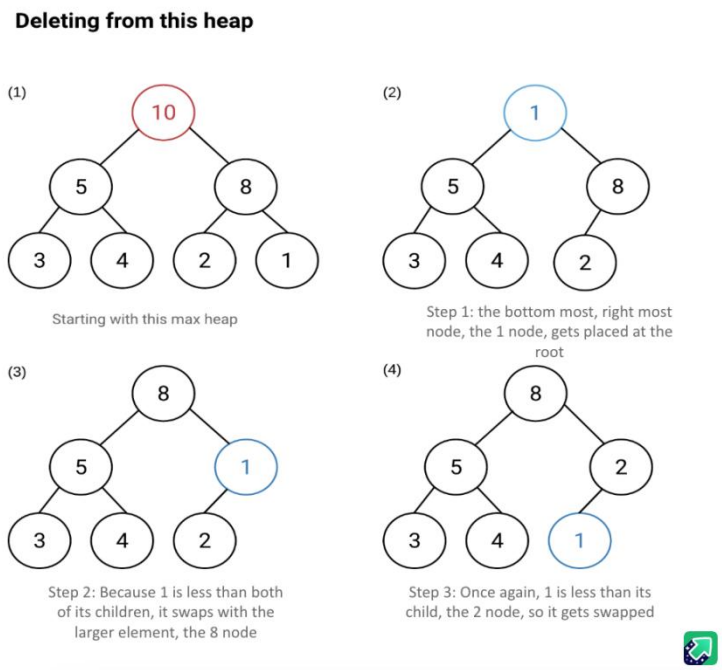
- CPU process scheduling
- Printer job scheduling
- Task management systems

Example:
If priorities are:

Task A – Priority 5
Task B – Priority 9
Task C – Priority 3

Using **Max Heap**, execution order:

Task B → Task A → Task C



****

```
#include <iostream>
using namespace std;

class MaxHeap {
    int heap[50];
    int size;

public:
    MaxHeap() {
        size = 0;
    }
```

```
    void insert(int value) {
        size++;
        int i = size; // first index position is 1

        // Insert at last position
        heap[i] = value;

        // Heapify Up
        while (i > 1 && heap[i] > heap[(i-1)/2]) {
            swap(heap[i], heap[(i-1)/2]);
            i = (i-1) / 2;
        }
    }
void deleteMax() {
if(size == 0) {
cout << "Heap is empty\n";
return;
}

cout << "Deleted element: " << heap[1] << endl;

// Replace root with last element
heap[1] = heap[size];
size--;

int i = 1;
```

```
// Heapify Down
while (2*i <= size) {
int largest = i;
int left = 2*i;
int right = 2*i + 1;

if (left <= size && heap[left] > heap[largest])
largest = left;

if (right <= size && heap[right] > heap[largest])
largest = right;

if (largest != i) {
swap(heap[i], heap[largest]);
i = largest;
} else {
break;
}
}
}


    void display() {
        cout << "Max Heap elements are: ";
        for(int i = 1; i <= size; i++) {
            cout << heap[i] << " ";
        }
        cout << endl;
    }
};
```

```
int main() {
    MaxHeap h;

    h.insert(50);
    h.insert(30);
    h.insert(40);
    h.insert(10);
    h.insert(5);
    h.insert(20);
    h.insert(60);

    h.display();

h.deleteMax();

cout << "Heap after deletion:\n";
h.display();
    return 0;
}
```